

Client/Institute Name : KIOT – SDET 2026
Course : Core Python
Technical Assessment : 21
Module : Basics Of Python
Date : 03-Jun-26
Time : 02:15 P.M – 05:15 P.M
Duration : 3 h
Total Marks : 50 Marks

Part A – 1 x 10 = 10 Marks

Part B — Programming Questions (40 Marks)

Write well-structured Python programs with comments, exception handling, and meaningful identifiers where applicable.

Q1. Student Grade Analyser

10 Marks

A training institute conducts an end-of-batch assessment for SDET trainees. The administrator enters each trainee's USN and marks (out of 100). The system must analyse the results and produce a structured report.

Program Requirements:

- Accept USN–marks pairs as a dictionary (hardcoded or user-input). Use the sample dataset below.
- Display the maximum and minimum marks along with the corresponding USN(s).
- Categorise and display count and USNs for: Distinction (86–100), Merit (76–85), Pass (60–75), Fail (below 60).
- Compute the class average. Identify trainees who scored below average.
- Sort and display the final leaderboard (USN: Marks) in descending order of marks.

Sample Input (Dictionary)	Expected Output (partial)
<pre> {"25MCA001":77, "25MCA009":60, "25MCA025":99, "25MCA007":84, "25MCA012":45, "25MCA021":86, "25MCA032":83, "25MCA018":40, </pre>	<pre> Maximum: 99 – 25MCA025 Minimum: 40 – 25MCA018 Distinction: 2 → [25MCA025, 25MCA021] Merit: 3 → [25MCA001, ...] </pre>

```
"25MCA014":67}
```

```
Pass: 2 | Fail: 2
```

```
Class Average: 71.22
```

```
Below Avg: [25MCA009, 25MCA012, 25MCA018]
```

```
--- Leaderboard ---
```

```
25MCA025:99 25MCA021:86 ...
```

Q2. Trainee Performance Hierarchy (OOP)

10 Marks

A company tracks training performance using an object-oriented model. Implement the class hierarchy below and compute each trainee's aggregate performance score.

Class Design:

- i. Person (Base class): attributes — name, age, email. Method: display_info().
- ii. Trainee (inherits Person): adds batch_id, marks (list of 5 subject scores), num_projects, num_publications. Overrides display_info().
- iii. SDETrainee (inherits Trainee): adds tool_proficiency (e.g. Selenium). Implements compute_aggregate() using the formula:

```
aggregate = (avg_marks × 0.6) + (num_projects × 5) + (num_publications × 3)
```

- iv. Read all attributes from user input. Create at least 2 objects. Display full info and aggregate for each.
- v. Identify and print the trainee with the highest aggregate score.

Sample Output — One Trainee

```
Name: Arun Kumar | Age: 24
Batch: B2025
Marks: [78,85,90,72,88] Avg:82.6
Projects: 3 | Publications: 1
Tool: Selenium
Aggregate Score: 67.56
```

Formula Breakdown

```
82.6 × 0.6 = 49.56
3 × 5      = 15.00
1 × 3      =  3.00
-----
Total      = 67.56
```

Q3. Log File Analyser

10 Marks

SDETs routinely parse server log files to identify anomalies. Write a Python program that analyses a log file, extracts meaningful statistics, and saves results to a report file.

Program Requirements:

- i. Read server_log.txt. Each line format: [LEVEL] YYYY-MM-DD HH:MM:SS — Message text
- ii. Count and display: total lines, total words, total characters, total vowels in the file.
- iii. Using regex, extract and count occurrences of each log level: INFO, WARNING, ERROR, CRITICAL.
- iv. Find and display all lines containing ERROR or CRITICAL as an alert summary.
- v. Write all results to log_report.txt. Implement proper try-except-finally for all file operations.

Sample Input (server_log.txt)	Sample Output (log_report.txt)
<pre>[INFO] 2025-06-01 09:00:01 — Service started [WARNING] 2025-06-01 09:05:12 — High memory [ERROR] 2025-06-01 09:10:44 — DB connection failed [INFO] 2025-06-01 09:15:00 — Request processed [CRITICAL] 2025-06-01 09:20:33 — Disk full</pre>	<pre>Total Lines : 5 Total Words : 35 Total Chars : 220 Total Vowels : 68 INFO:2 WARNING:1 ERROR:1 CRITICAL:1 --- ALERTS --- [ERROR] ... DB connection failed [CRITICAL] ... Disk full</pre>

Q4. API Test Data Processor (Modular Design)
10 Marks

In a real-world SDET project, test data for API testing is stored in a structured format. Build a modular Python application that validates, processes, and categorises trainee registration data submitted via a mock API payload.

Module: data_validator.py

- i. Define validate_email(email) using regex. Must match: user@domain.ext (extension: 2–5 alpha chars).
- ii. Define validate_phone(phone): Indian mobile — starts with 6–9, exactly 10 digits.
- iii. Define validate_usn(usn): pattern 25MCA\d{3} e.g. 25MCA007.
- iv. Raise a custom exception InvalidFieldError(field_name, value) for any validation failure.

Module: data_processor.py

- v. Accept n trainee records as tuples: (usn, name, email, phone). Store as tuple-of-tuples (immutable).

- vi. For each record, extract username (before @) and domain (after @). Store as named tuples.

Main Script: main.py

- vii. Import both modules. Accept n records from user input. Run full validation before processing. Display a structured summary table.
- viii. Gracefully handle all `InvalidFieldError` exceptions — log the error and continue to next record.

Sample Input	Sample Output
<pre>Enter records: 2 USN: 25MCA001 Name: Kavya Email: kavya@smartcliff.in Phone: 9876543210 --- USN: 25MCA002 Name: Rajan Email: rajan@gmail.com Phone: 9123456780</pre>	<pre>Valid Records: 2 USN Name Username Domain 25MCA001 Kavya kavya smartcliff.in 25MCA002 Rajan rajan gmail.com Stored as immutable tuple-of-tuples.</pre>